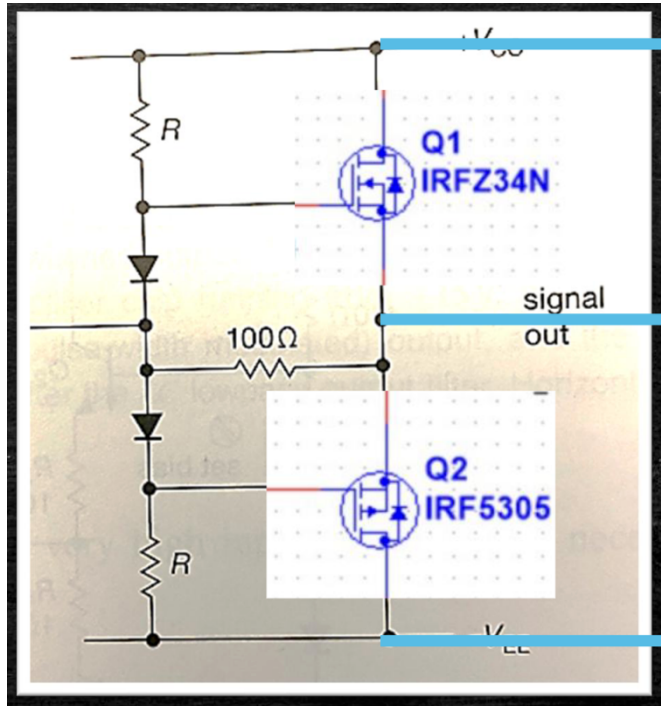


Operational Amplifier Using PNP and NPN MOSFET's



← this MOSFET is a N-channel MOSFET

<- this is a P-Channel MOSFET

The Left (N-Channel MOSFET): You will see the **IRFZ34N** here.

- **Logic:** It is acting as a "Low-Side" switch. The arrow on the symbol points **IN** toward the gate.
- **Placement:** It sits between the Load and the Ground. When the Gate gets a "High" signal, it closes the circuit to ground.

The Right (P-Channel MOSFET): This is where the **IRF5305** is positioned.

- **Logic:** It acts as a "High-Side" switch. The arrow points **OUT** toward the source.
- **Placement:** It sits between the Power Supply (V_{CC}) and the Load. It turns ON when the Gate is pulled "Low" relative to the source.

2 options:

1. Switching Amplification (Class D / PWM)

This is the most common use in Simulink projects. Instead of varying the voltage smoothly, you flip the MOSFET on and off very fast (Pulse Width Modulation).

- **The Gain:** You are amplifying the *capability* of your control signal. A microcontroller cannot provide the 100 Watts a motor needs, but the MOSFET "amplifies" that control signal by tapping into the main power supply to provide that 100 Watts.

- **Efficiency:** This is highly efficient because the MOSFET is either "fully on" (low resistance) or "fully off" (no current), so it doesn't waste much power as heat.
- **2. Linear Amplification**
- If you operate the MOSFET in its **Saturation Region** (also called the Active Region), it acts as a true analog amplifier.
- A small change in the Gate voltage results in a large, proportional change in the Drain current .
- This is how audio amplifiers work to make a tiny phone signal loud enough to vibrate a massive speaker.

The diodes act as a safety mechanism. By providing a path for inductive kickback, they ensure the 'power amplification' doesn't lead to voltage spikes that would exceed the MOSFET's breakdown voltage ().

How it will work

The Raspberry Pi is the "brain," but it has very "weak" muscles.

- **The Pi's Limit:** Each GPIO pin can only safely provide about **16mA** of current at **3.3V**.
- **The System's Need:** the system likely needs **Amps** of current at **12V or more**.
- **The MOSFET's Job:** It acts as a **Relay without moving parts**. The Pi's 3.3V signal creates an electrostatic field that "unlocks" the high-voltage battery to flow into your system.

2. The Detailed "Flow" from the Pi

Let's look at the flow from the Pi's GPIO pin through the MOSFETs:

Left Side: N-Channel (Low-Side Switch)

This is the most common way to use a Pi with a MOSFET.

1. **SIGNAL:** The Pi sets a GPIO pin to **HIGH (3.3V)**.
2. **GATE:** This voltage hits the Gate of the **IRFZ34N**.
3. **CHANNEL:** Electrons are pulled into the "Channel" inside the silicon, making it conductive.
4. **FLOW:** Your system's "Ground" is now connected to the actual Battery Ground. Current flows from the Battery (+) through your system into the MOSFET Drain out the Source to Ground.

5. **THE CAVEAT:** The IRFZ34N is **not** a "Logic Level" MOSFET. At 3.3V from a Pi, it might not open "all the way." It might get very hot because its resistance () stays high.

Right Side: P-Channel (High-Side Switch)

This side is trickier with a Pi.

1. **LOGIC:** To turn a P-channel **ON**, the Gate must be **0V** (Low). To turn it **OFF**, the Gate must be the **same as the Source voltage** (e.g., 12V).
2. **THE PROBLEM:** If your battery is 12V, the Pi can only output 3.3V. Even when the Pi is "High," the MOSFET thinks it's "Low" relative to 12V and stays **ON**.
3. **THE SOLUTION:** Usually, people use the **N-channel (Left)** to "pull down" the Gate of the **P-channel (Right)**. This is a "driver" stage that allows the 3.3V Pi to control 12V+ power.

When the Pi tells the MOSFET to turn off, the flow in your system collapses.

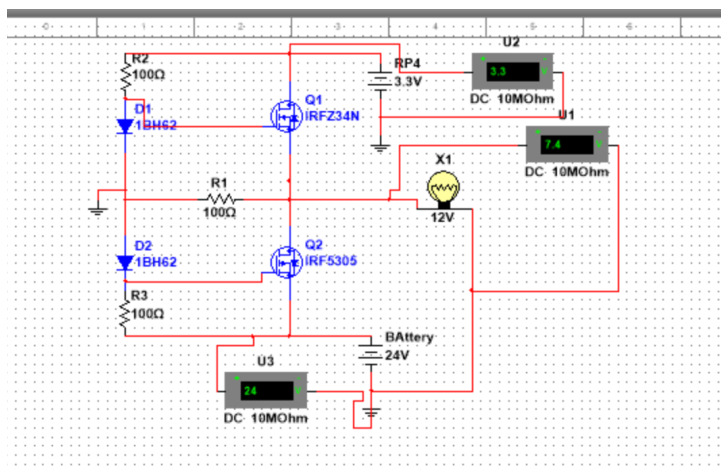
- **Without the Diode:** A 12V+ spike could jump back through the MOSFET and **fry your Raspberry Pi's CPU instantly**.
- **With the Diode:** The spike is trapped in a loop at the motor/load and never reaches the Pi.

I got tripped up thinking these MOSFETS amplify the RP4 signal, but really with a GPIO at 3.3V it only unleashes the potential from the battery, the MOSFET's and diodes keep the raspberry pi safe. The 3.3 V from the RP4 is enough to get through the first MOSFET which then signals the battery to power the load.

Multisim:

We should aim to load about 12-20 V out of our battery to the TL081's

Initially I can only get 7.4 V to exit the system



The reason you are seeing is that your current design is acting as a **Voltage Follower**. In this setup, the output voltage is "clamped" by the gate voltage minus the threshold voltage (V_{th}). Since your Pico only provides V_{Pico} , the MOSFETs can't "climb" any higher.

To get to using only the components in your image, you need to change **how the MOSFETs are triggered**.

1. The "Common Source" Configuration (The Fix)

In your current Multisim build, the "Load" (the lightbulb) is on the **Source** side of the N-channel. This is the hardest way to get high voltage. To get the full battery voltage, you need to move the load so the MOSFETs act as **Common Source** switches.

- **For the N-Channel (Q1):** Connect the **Source** directly to Ground and put the **Load** between the Drain and the rail.
 - **For the P-Channel (Q2):** Connect the **Source** directly to and put the **Load** between the Drain and Ground.
-

2. Why the "Center" Signal Out is Limited

In your diagram, the output is taken from the middle of the two MOSFETs. This is a "Half-Bridge."

- Because the Pico is only V_{Pico} , the **IRFZ34N** (Top) can only pass voltage until its Source reaches about V_{th} (its threshold).
- The you are seeing is likely the result of the **Voltage Divider** created by the resistors (R_1 , R_2) and the internal resistance of the MOSFETs while they are partially "choked".

3. How to satisfy the Teacher's Requirements

If you must keep this specific layout but want more voltage, you have to increase the **input signal voltage** before it hits the MOSFETs. If you can't use an Op-Amp, you can use a **Discrete Transistor Driver**:

1. **Use a small NPN transistor** (like a 2N2222).
2. Let the **Pico (3.3V)** turn on the NPN.
3. Let the NPN pull the MOSFET Gates down from the **rail**.

This allows the Gates to see the full swing of the battery instead of being stuck at the V_{Pico} of the Pico.